CSE3001 – Database Management Systems

Module 1 Notes — Introduction to DBMS, Architecture, SQL Languages & ER Modeling
Compiled from lecture material by Prof. Rajneesh Kumar Patel, Asst. Professor, CSE

Contents

1. Data, Database & DBMS
2. File System vs Database System
3. DBMS Architecture (Three-Tier & Structure)
4. Three-Level Schema Architecture & Data Independence
5. Database Schemas & Instances
6. Database Languages — DDL, DML, DCL, TCL (with SQL examples)
7. Functions of a DBA
8. Data Models & their Evolution
9. Entity-Relationship (ER) Model — Entities, Attributes, Relationships
10. Participation Constraints & Recursive Relationships
11. Steps to Draw an ER Diagram
12. Extended ER Features — Aggregation, Generalization, Specialization
13. Converting an ER Model into a Relational Model

1. Data, Database & DBMS

1.1 Data

Data refers to a collection of raw, unprocessed facts. These are facts that can be recorded and that carry an implicit meaning — for example, a student's roll number, name, and address. By itself, data is just raw material; it becomes useful information only once it is organized and interpreted.

1.2 Database

A database is a collection of interrelated data, typically stored in the form of tables. The data inside it can be stored, updated, or deleted either manually or through a computerized system, and the records are logically connected to one another (e.g., a Student table linked to a Department table).

1.3 Database Management System (DBMS)

A DBMS is a collection of interrelated data together with a set of programs that allow users to easily store, update, and retrieve that data. In short: **Interrelated Data + Set of Programs (Instructions) = DBMS**. Its primary goal is to let operations be performed on data without anomalies such as redundancy, inconsistency, or loss of integrity.

2. File System vs Database System

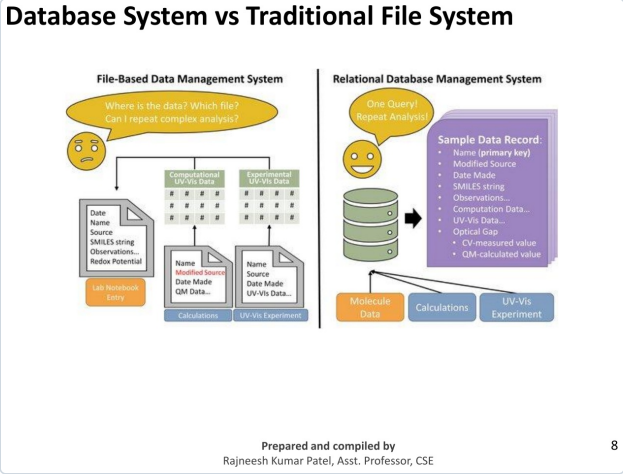


Fig. Traditional file-based data management vs a Relational Database Management System

2.1 Disadvantages of File Processing Systems

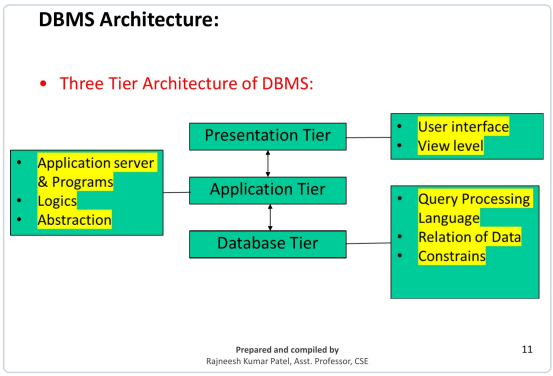
- **Data Redundancy** — the same data gets duplicated across multiple files.
- **Difficulty in accessing data** — no unified way to query across files.
- **Data Isolation** — related data scattered in several files/formats, hard to combine.
- **Atomicity problems** — no guarantee of "all or none" execution of operations.
- **Concurrency control anomalies** — no built-in mechanism for simultaneous access.
- **Poor security** — data is spread out and hard to protect uniformly.
- **Weak integrity constraints** — domain constraints, key constraints, and referential integrity are difficult to enforce.

2.2 File-Based Approach vs Database Approach

#	File-Based Approach	Database Approach
1	Program and data are interdependent.	Program and data are independent of each other.
2	Data may be duplicated across files, causing redundancy.	Data is not duplicated — it appears only once.
3	Same data in different files may differ, causing inconsistency.	Since data appears once, it stays consistent.
4	Data is scattered, making retrieval difficult.	Data is stored in linked tables; applications retrieve it easily.
5	Data is distributed across files and cannot be shared.	Data is centralized and easily shared.
6	Difficult to apply data-integrity checks.	Provides many built-in constraints for integrity.
7	Poor security since data is widely spread.	Provides many procedures to maintain security.
8	Difficult to maintain — few controlling facilities.	Provides many facilities to maintain programs easily.
9	Less complex system.	Very complex system.
10	Cost is much less than a DBMS.	Cost is much higher than a file system.
11	If one application fails, others are unaffected.	All applications relying on the database fail if the database fails.

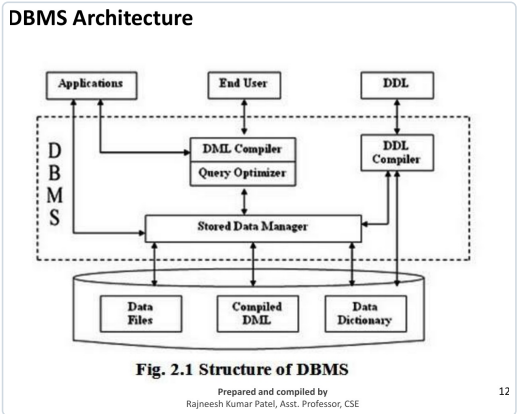
3. DBMS Architecture

3.1 Three-Tier Architecture of DBMS



- **Presentation Tier** — the user interface / view level, where the end user interacts with the system.
- **Application Tier** — contains the application server & programs, business logic, and abstraction.
- **Database Tier** — handles query-processing language, relations of data, and constraints.

3.2 Structure of DBMS



> Inside the DBMS, the **DML Compiler** (with its Query Optimizer) handles data-manipulation requests from applications and end users, while the **DDL Compiler** handles schema-definition statements. Both route through the **Stored Data Manager**, which interacts with the physically stored Data Files, Compiled DML, and Data Dictionary on disk.

4. Three-Level Schema Architecture & Data Independence

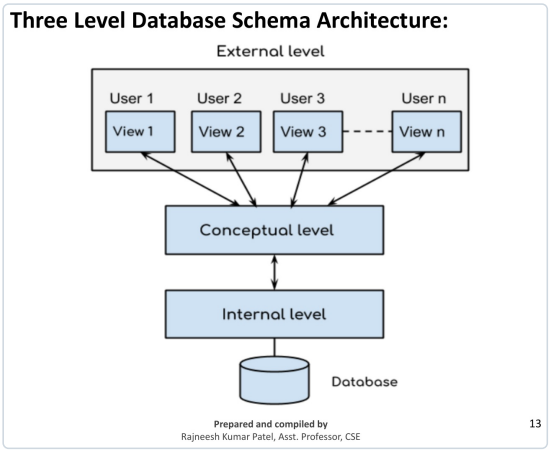


Fig. External, Conceptual and Internal levels of a database

4.1 The Three Levels

(a) Internal Level

- Also called the **physical schema**; uses a physical data model.
- Describes the physical storage structure of the database.
- Defines how the data will be stored in a block.

(b) Conceptual Level

- Also called the **logical level**; describes the design of the database at a conceptual level.
- Describes what data is stored and what relationships exist among the data.
- Hides internal implementation details such as the physical data structure.
- This is the level where programmers and database administrators work.

(c) External Level

- Also called the **view schema**; describes different views of the database.
- Each view describes the part of the database that a particular user group is interested in, hiding the rest.
- Represents the end user's interaction with the database system.

4.2 Data Independence

> A database system that can separate these three levels of abstraction becomes flexible and adaptable — this property is called **Data Independence**. It means that a change at a lower level should not affect the level above it.

- **Physical Data Independence** — separation of the Conceptual and External views from the Internal view; the physical structure of the database can change without affecting the conceptual/external levels.
- **Logical Data Independence** — separation of the External view from the Conceptual view; changes to the conceptual schema (e.g., adding a new entity) do not affect existing external views.

5. Database Schemas & Instances

5.1 Database Schema

> A schema is the description of the database — its overall design. It is specified during database design and is not expected to change frequently.

```
Student (Roll_No Varchar(10), Name Char(30), Address varchar(30), Phone_No Number(10), DOB date...)
Department (Dept_Id Varchar(10), Dept_Name Char(10), Course_Name Char(20)...)
```

5.2 Database Instance

> An instance is the collection of information stored in the database at a particular moment in time — essentially a "snapshot" of the database.

Roll No	Name	Contact No	Address
101	Ajay	123	BPL
125	Kartik	456	BPL
140	Rahul	789	BPL

6. Database Languages

6.1 DDL — Data Definition Language

> DDL is used to define the database structure or pattern — creating schemas, tables, indexes, and constraints. It stores metadata such as the number of tables and schemas, their names, indexes, columns, and constraints.

- **CREATE** — creates objects in the database.
- **ALTER** — alters the structure of the database.
- **DROP** — deletes objects from the database.
- **TRUNCATE** — removes all records from a table.
- **RENAME** — renames an object.

```
SQL> create table student_detail(roll_no number(10), name char(30));
Table created.

SQL> alter table student_detail add(contact_no number(10), address varchar(20));
Table altered.

SQL> alter table student_detail modify(roll_no varchar(20) primary key);
Table altered.

SQL> alter table student_detail drop(contact_no);
Table altered.

SQL> rename student_detail to student;
Table renamed.

SQL> desc student;
Name                          Null?     Type
----------------------------- --------  ----------------------------
ROLL_NO                       NOT NULL  VARCHAR2(20)
NAME                                    CHAR(30)
ADDRESS                                 VARCHAR2(20)

SQL> truncate table student;      Table truncated.
SQL> drop table student;          Table dropped.
```

6.2 DML — Data Manipulation Language

> DML is used for accessing and manipulating data already stored in the database; it handles user requests for reading and modifying data.

- **SELECT** — retrieves data from the database.
- **INSERT** — inserts data into a table.
- **UPDATE** — updates existing data within a table.
- **DELETE** — deletes records from a table.

```
SQL> insert into student values ('CS101', 'Ajay', 'Bhopal');
SQL> insert into student values ('CS102', 'Aman', 'Ujjain');
SQL> insert into student (roll_no) values ('CS104');
SQL> insert into student (name) values ('Anuj');
* ERROR at line 1: ORA-01400: cannot insert NULL into ("ROLL_NO")  -- Roll_No is NOT NULL

SQL> select * from student where address = 'Bhopal';
SQL> select roll_no as roll_number from student;      -- column aliasing

SQL> update student set address='Bhopal' where roll_no='CS103';
SQL> update student set name='Anay', address='Ujjain' where roll_no='CS104';

SQL> delete from student where roll_no='CS104';        1 row deleted.
SQL> delete from student where address='Bhopal';       2 rows deleted.
```

> **Note:** Because `ROLL_NO` was set as `NOT NULL` (a PRIMARY KEY via ALTER), inserting a row without a roll number raises `ORA-01400`.

6.3 DCL — Data Control Language

> DCL is used to control access to data stored in the database — essentially managing user permissions.

- **GRANT** — gives a user access privileges to the database.
- **REVOKE** — takes back permissions from a user.

6.4 TCL — Transaction Control Language

> TCL is used to manage the changes made by DML statements; these statements are grouped into logical transactions.

- **COMMIT** — saves the transaction to the database permanently.

- **ROLLBACK** — restores the database to its state at the last COMMIT.

7. Functions of a Database Administrator (DBA)

- Designing the Conceptual & Physical Schema.
- Security & Authorization management.
- Ensuring Data Availability & Recovery from failures.
- Defining Storage Structures and Access Methods.
- Specifying Integrity Constraints.

8. Data Models & their Evolution

> A data model is a collection of concepts used to describe the structure of a database — its data types, relationships, and constraints. It specifies the dynamic aspects and behaviour of database applications, offers relatively simple (usually graphical) representations of complex real-world structures, and facilitates interaction among the designer, application programmer, and end user.

8.1 Three Levels of Modeling (analogy: Civil Engineering)

In Civil Engineering:

- Final project (built structure)
- Charts / Blueprint (representation)
- High-level / conceptual view (idea/design)

In DBMS:

- **High-level / Conceptual view** → ER Model
- **Representation / Implementation** → Relational Model (Tables — what programmers use via SQL)
- **Low-level / Physical data model** → How tables are stored in memory: structure of records, data types, data retrieval

8.2 Evolution of Data Models

- **High Level / Object-Based Data Model**
  - Entity-Relationship (ER) Model
  - Object-Oriented Data Model
- **Representation / Record-Based Data Model**
  - Relational Model
  - Network Model
  - Hierarchical Model
- **Low Level / Physical Data Model**
  - Unifying Model
  - Frame Memory Model

8.3 Entity-Relationship (ER) Model — Overview

- A widely accepted, graphical tool for data modeling.
- A high-level conceptual data-model diagram that helps analyze data requirements to produce a well-designed database.
- Graphically represents entities and their relationships in a database structure, explaining its logical structure.
- **Components:** Entity, Relationship, Attributes.

8.4 Object-Oriented Data Model (OODM)

- Models both data and their relationships in a single structure called an **object**.
- Forms the basis for Object-Oriented Database Management Systems (OODBMS).
- The object becomes the basic building block for autonomous structures.
- **Object** — an abstraction of a real-world entity; **Attributes** describe its properties.
- Objects sharing similar characteristics are grouped into **Classes**.
- **Inheritance** — the ability of an object within a class hierarchy to inherit attributes and methods of classes above it.

8.5 Relational Model

- Developed by **Codd (IBM) in 1970**; conceptually simple.
- Forms the extension for the Relational Database Management System (RDBMS).
- Its greatest advantage is hiding the complexities of the relational model from the user.
- Represents a database's entities, their attributes, and the relationships between entities.

> **Relational Table (relation):** stores a collection of related entities as a matrix of row/column intersections. It is a purely logical structure — how data is physically stored is of no concern to the user or designer. This property triggered a real database revolution.

> **SQL (Structured Query Language):** lets users specify *what* must be done, without specifying *how*. A SQL-based relational application involves a user interface, a set of tables stored in the database, and a SQL engine.

8.6 Hierarchical Model

- Developed in the **1960s** to manage large amounts of data for complex manufacturing projects.
- Basic logical structure represented by an upside-down "tree".
- Structure contains levels or segments.
- Depicts a set of **one-to-many (1:M)** relationships between a parent and its children's segments.

Advantages: many of its features formed the foundation of current data models; its database-application advantages are still replicated in modern environments.

Disadvantages: complex to implement, difficult to manage, and has implementation limitations.

8.7 Network Model

- Developed to represent complex data relationships more effectively.
- Improves database performance with the help of links/connections.
- Represents a collection of records connected via **M:M (many-to-many)** relationships.

9. Entity-Relationship (ER) Model — In Detail

> The ER Model represents real-world entities and the relationships between them in a high-level conceptual diagram. It helps analyze data requirements to produce a well-designed database and explains the logical structure of a database. Its three components are **Entity, Attribute,** and **Relationship**.

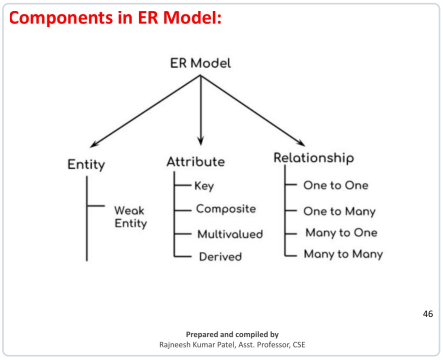
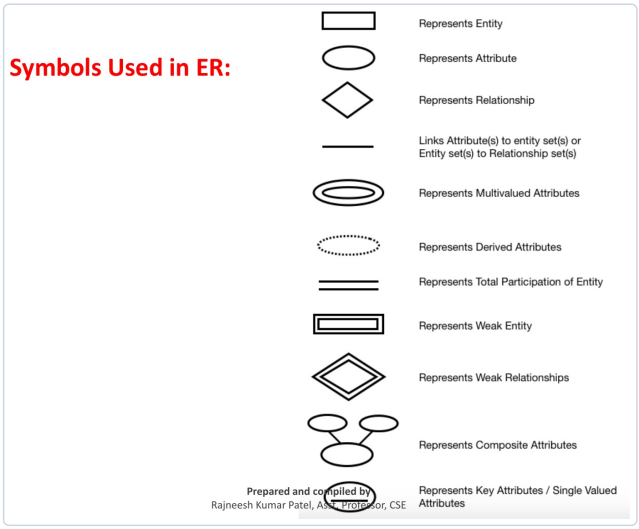


Fig. Components in the ER Model

9.1 Symbols Used in ER Diagrams



Symbol	Represents
Rectangle	Entity
Oval	Attribute
Diamond	Relationship
Straight line	Links attribute(s) to entity set(s), or entity set(s) to relationship set(s)
Double oval	Multivalued attribute
Dashed oval	Derived attribute
Double line	Total participation of an entity
Double rectangle	Weak entity
Double diamond	Weak relationship
Connected ovals	Composite attribute
Underlined oval	Key attribute / single-valued attribute

9.2 Entity & Entity Types

- An entity may have a **physical existence** — a person, car, house, or employee.
- An entity may have a **conceptual existence** — a company, a job, or a university course.
- An entity is an object of an entity type; the set of all entities is called an **entity set**.
- An entity is represented as a rectangle in an ER diagram.

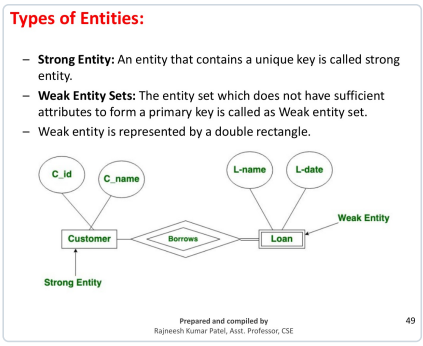


Fig. Strong Entity (Customer) and Weak Entity (Loan) in a Borrows relationship

- **Strong Entity** — an entity that contains a unique key attribute (its own primary key).
- **Weak Entity** — an entity set that does not have sufficient attributes to form its own primary key; it depends on a strong (owner) entity and is represented by a **double rectangle**.

9.3 Attributes & their Types

> An attribute describes a property of an entity and is represented as an oval in an ER diagram. There are four broad categories of attributes.

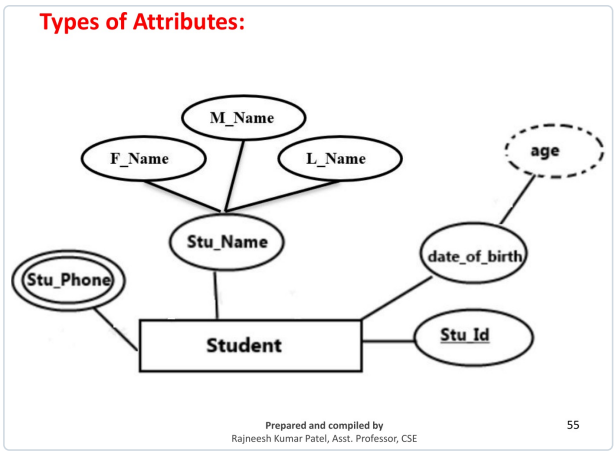


Fig. Simple, composite, multivalued, key and derived attributes of the Student entity

1. **Key Attribute** — uniquely identifies an entity within an entity set; represented as an oval with the text underlined (e.g., RollNo).
2. **Simple & Composite Attribute** — a **composite** attribute is a combination of other attributes (e.g., Address = Pin + State + Country); a **simple** attribute cannot be divided further.
3. **Single-valued & Multivalued Attribute** — a **multivalued** attribute can hold multiple values (e.g., a person's Phone_No) and is drawn with a **double oval**.
4. **Stored & Derived Attribute** — a **derived** attribute's value is dynamic and computed from another attribute (e.g., Age derived from DOB); it is shown with a **dashed oval**.

9.4 Relationships in the ER Model

> A relationship is the association among entities, represented by a diamond shape in an ER diagram. There are four cardinality types of relationships.

(a) One-to-One (1:1)

When a single instance of an entity is associated with a single instance of another entity. *Example: a person has only one passport, and a passport is issued to only one person.*

(b) One-to-Many (1:M)

When a single instance of an entity is associated with more than one instance of another entity. *Example: a customer can place many orders, but an order cannot be placed by many customers.*

(c) Many-to-One (M:1)

When more than one instance of an entity is associated with a single instance of another entity. *Example: many students can study in a single college, but a student cannot study in many colleges at once.*

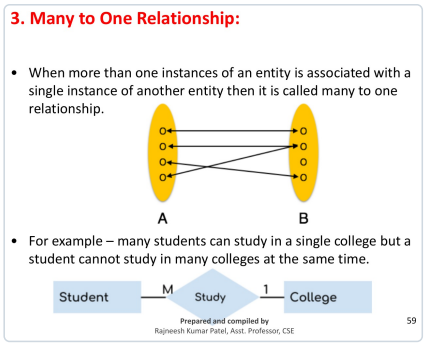


Fig. Many-to-One relationship (Student — Study → College)

(d) Many-to-Many (M:M)

When more than one instance of an entity is associated with more than one instance of another entity. *Example: a student can be assigned to many projects, and a project can be assigned to many students.*

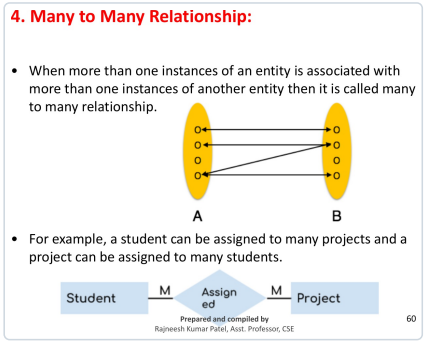
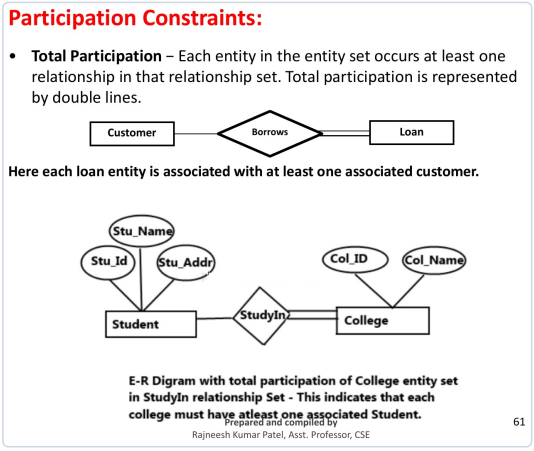


Fig. Many-to-Many relationship (Student — Assigned → Project)

10. Participation Constraints & Recursive Relationships

10.1 Participation Constraints



- **Total Participation** — every entity in the entity set occurs in at least one relationship instance in that relationship set. It is drawn with a **double line**. Example: every Loan entity must be associated with at least one Customer (via "Borrows").
- **Partial Participation** — an entity in the entity set may or may not occur in a relationship instance. Example: every Department must be managed by at least one Employee (total participation of Department), but not every Employee manages a department (partial participation of Employee).

10.2 Recursive Relationship

> A recursive relationship occurs when the same entity participates more than once in a relationship. *Example: an Employee entity that can play the role of both "Manager" and "Worker" in a "WorksFor" relationship with itself.*

11. Steps to Draw an ER Diagram

Problem statement: In a university, a Student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a Professor can deliver only one course.

Step 1 — Entity Identification

Identify the entities: **Student, Course, Professor.**

Step 2 — Relationship Identification

Identify the relationships: the Student is **assigned** a Course; the Professor **delivers** a Course.

Step 3 — Cardinality Identification

Determine cardinalities from the statement: many students can be assigned to multiple courses (M:N); a professor can deliver only one course (1:1).

Step 4 — Identify Attributes

Identify the attributes for each entity without initially mapping them, then pair each attribute with exactly one entity (use a modifier if an attribute could belong to more than one entity).

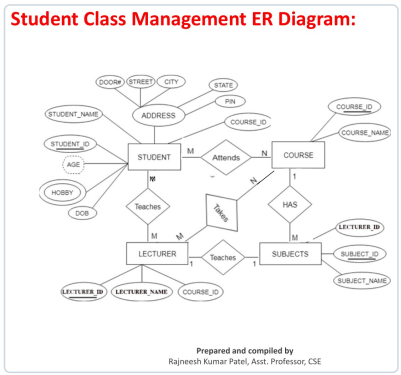
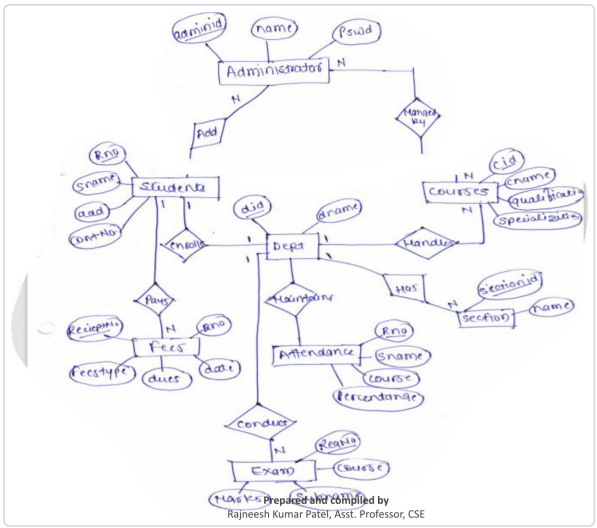


Fig. Completed ER diagram — Student, Course, Professor with cardinalities and key attributes

Step 5 — Create the ER Diagram

Combine entities, relationships, cardinalities, and attributes into the final ER diagram (shown above).

Worked example: A larger, hand-drawn ER diagram for a college management system (Administrator, Students, Courses, Department, Fees, Attendance, Exam, Section) is shown below to illustrate how multiple entities and relationships combine in a real system.



12. Extended ER Features

12.1 Aggregation

Aggregation represents a "relationship among a relationship" — it is used to express a relationship between a whole object and its component part. It allows a relationship set to participate in another relationship set, acting as an abstraction for building composite objects out of their component objects. It is drawn using a dashed line/box around the inner relationship in an ER diagram.

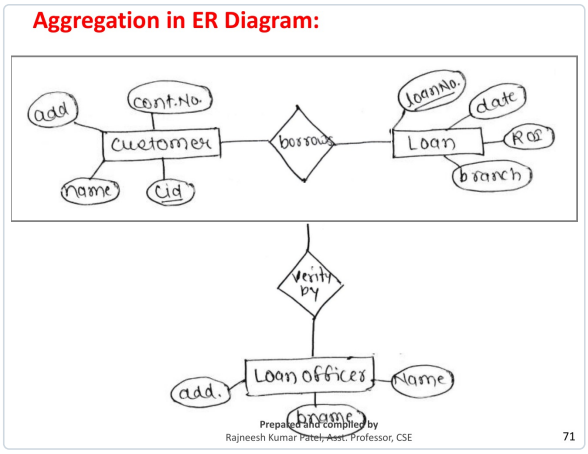


Fig. Aggregation — the (Customer–Borrows–Loan) relationship is treated as a single object, verified by a Loan Officer

12.2 Generalization

> Generalization is a **bottom-up** approach where two or more lower-level entities (with their own attributes) combine to form a single higher-level (generalized) entity, based on their common attributes. For example, Employee and Customer entities can both be generalized into a single "Person" entity.

12.3 Specialization

> Specialization is the reverse — a **top-down** approach where a higher-level entity is split into two or more lower-level, more specific entities. For example, an "MBBS" entity can be specialized into "Physician" (MD) and "Surgeon" (MS).

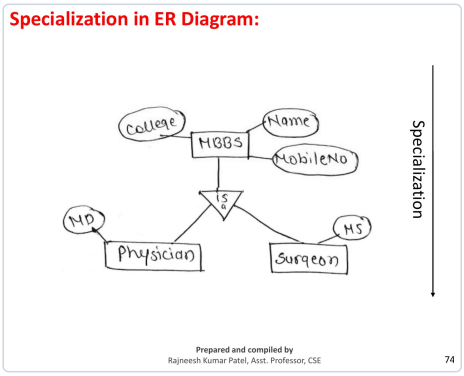


Fig. Specialization — MBBS specialized into Physician and Surgeon

12.4 Combined Generalization & Specialization

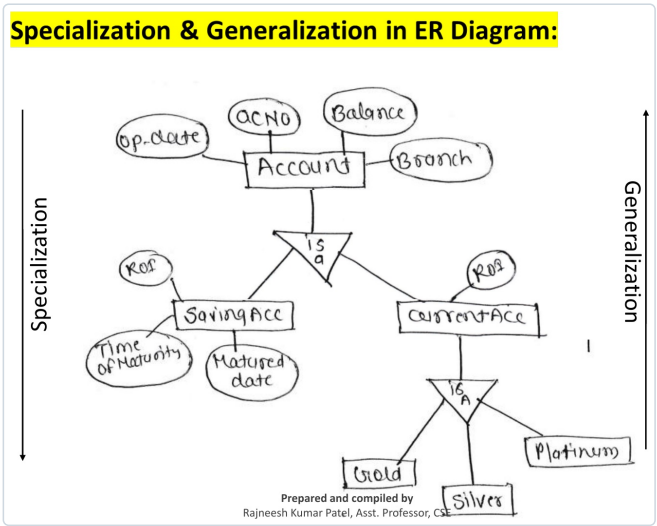


Fig. Account generalized/specialized into Saving Account and Current Account, further specialized into Gold, Silver, and Platinum

- The "is-a" triangle symbol is used for both generalization and specialization hierarchies.
- Reading bottom-up (from sub-types to a super-type) = **Generalization**.
- Reading top-down (from a super-type to sub-types) = **Specialization**.

13. Converting an ER Model into a Relational Model

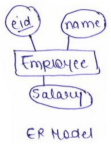
> Once an ER diagram is finalized, it must be mapped into a set of relational tables so it can be implemented in an RDBMS. The rules below cover the common cases.

13.1 Entity with a Key Attribute

A simple entity with a key and normal attributes maps directly to one table, with the key attribute becoming the **Primary Key**.

Conversion of ER Model into Relational Model

1. ER with Key Attribute:



Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000

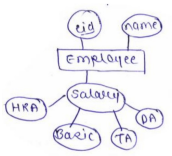
Prepared and compiled by
Rajneesh Kumar Patel, Asst. Professor, CSE
77

13.2 Entity Set with a Composite Attribute

A composite attribute (e.g., Salary = Basic + HRA + TA + DA) is "flattened" — each of its sub-parts becomes its own column in the table.

Conversion of ER Model into Relational Model

2. Entity set with Composite Attribute:



Employee					
Eid	**Name**	**Basic**	**HRA**	**TA**	**DA**
1201	Aman	32000	3000	1500	4500
1202	Prateek	29000	3000	1500	4500
1203	Bhavya	30000	3000	1500	4500

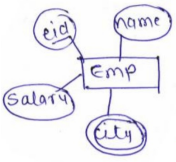
Prepared and compiled by
Rajneesh Kumar Patel, Asst. Professor, CSE

13.3 Entity with a Multivalued Attribute

A multivalued attribute (e.g., City, where an employee can have multiple cities) **cannot** stay in the main table as a single column — it must be split into a **separate table** containing the entity's key and the multivalued attribute, linked by a foreign key.

Conversion of ER Model into Relational Model

3. ER with Multivalued Attribute:



Employee			
Eid	**Name**	**Salary**	**City**
1201	Aman	56,000	Indore, Bhopal
1202	Prateek	49,000	Indore
1203	Bhavya	52,000	Bhopal

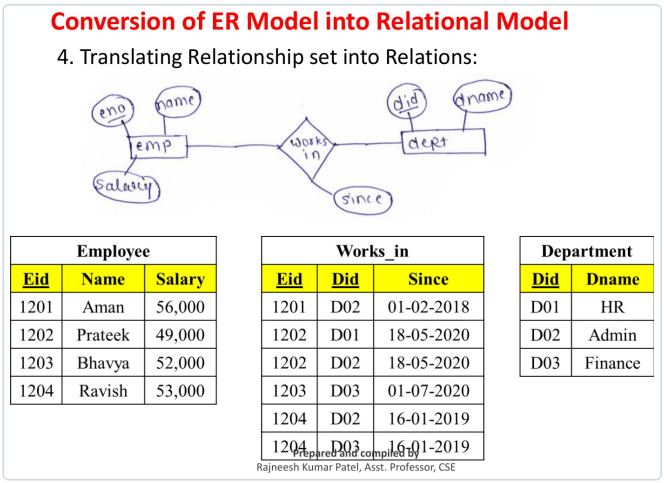
Employee		
Eid	**Name**	**Salary**
1201	Aman	56,000
1202	Prateek	49,000
1203	Bhavya	52,000

Employee	
Eid	**City**
1201	Indore
1201	Bhopal
1202	Indore
1203	Bhopal

Prepared and compiled by
Rajneesh Kumar Patel, Asst. Professor, CSE

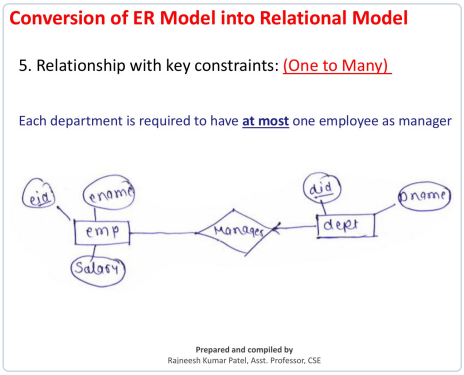
13.4 Translating a Relationship Set into Relations

A relationship set (e.g., "Works_in" between Employee and Department) becomes its own table containing the primary keys of both participating entities (as foreign keys) plus any relationship attributes (e.g., "Since").



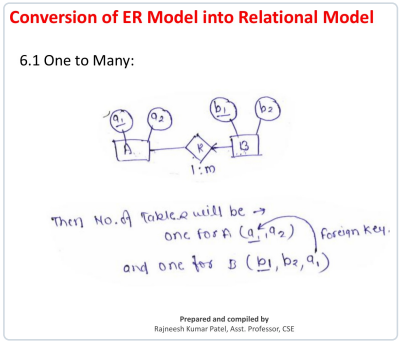
13.5 Relationship with Key Constraints (1:M)

For a **One-to-Many** relationship (e.g., a department has at most one employee as manager), the relation can be represented with a separate "Manages" table, or the foreign key can be merged into the "many" side / "one" side depending on the constraint, avoiding a redundant extra table.

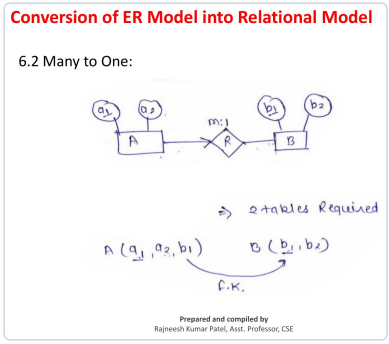


13.6 General Mapping Rules by Cardinality

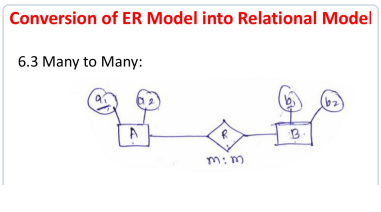
Relationship type	Minimum tables required	Mapping rule
One-to-Many (1:M) A(a1,a2) —R→ B(b1,b2)	2	Table A(a1,a2); Table B(b1,b2,a1) — the "one" side's key (a1) becomes a **foreign key** in the "many" side (B).
Many-to-One (M:1) A(a1,a2) —R→ B(b1,b2)	2	Table A(a1,a2,b1); Table B(b1,b2) — the "one" side's key (b1) becomes a **foreign key** in the "many" side (A).
Many-to-Many (M:M)	3	Since there is no key constraint, 3 tables are needed: A(a1,a2), B(b1,b2), and a separate relationship table R(a1,b1) holding both keys as a composite/foreign key.
One-to-One (1:1)	2 (max)	Either merge B's key into A → A(a1,a2,b1), B(b1,b2); or merge A's key into B → A(a1,a2), B(b1,b2,a1).

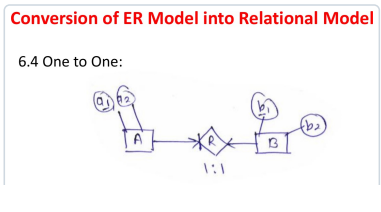


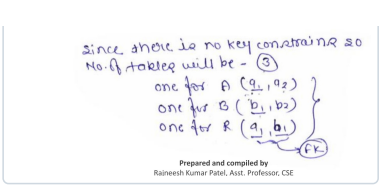
One-to-Many mapping



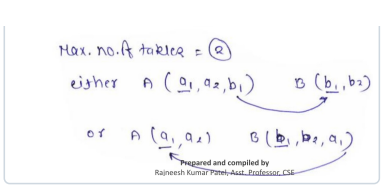
Many-to-One mapping







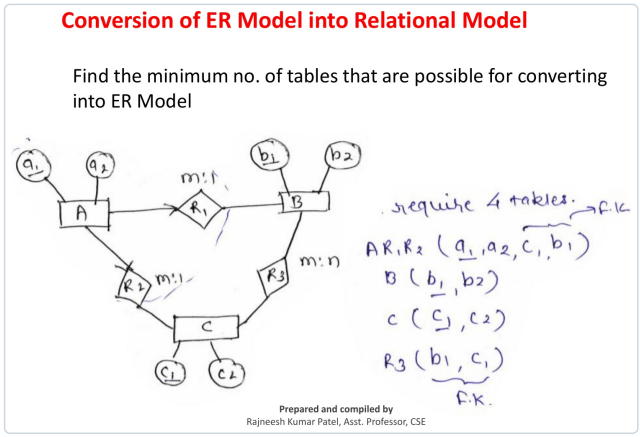
Many-to-Many mapping



One-to-One mapping

13.7 Worked Example — Minimum Number of Tables

For a chain of entities A, B, C connected through relationships R1 (M:1, A→B), R2 (M:1, A→C) and R3 (M:N, B→C), the minimum number of tables required is **4**: since A participates in two many-to-one relationships (R1 and R2), both foreign keys (from B and C) can be merged into A's own table.



A combined with R1 & R2 → A(a1,a2,c1,b1); B(b1,b2); C(c1,c2); R3(b1,c1) as a separate table (foreign keys)

Key takeaway for mapping ER → Relational:

- Every strong entity → one table, key attribute → primary key.
- Weak entity → its own table, including the owner entity's key as part of its primary key (foreign key).
- 1:1 and 1:M / M:1 relationships can often be merged into an existing entity's table using a foreign key — no separate relationship table is strictly needed.
- M:N relationships **always** require a separate table holding both participating entities' keys.
- Multivalued attributes always require a separate table.

— End of Module 1 Notes —

Compiled from lecture slides by Prof. Rajneesh Kumar Patel, Asst. Professor, CSE (CSE3001)